

OOP

programare orientată pe obiecte

CURS08: GENERICITATE

Alexandru Olteanu
{alexandru.olteanu@upb.ro}

Computer Science & Engineering Department
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Universitatea Politehnica București

O singura implementare mai multe tipuri de date:

Structuri de date (clase / interfete generice)

```
List <Player> team = new ArrayList <Player >() ;
List <Integer> grades = new ArrayList < Integer >() ;
```

Algoritmi (metode generice)

```
Collections . sort ( team ) ;
Collections . sort ( grades ) ;
```

De ce sa nu folosim pur si simplu un tip general de date (e.g. Object in Java)?

```
List myIntList = new LinkedList () ;
myIntList.add( new Integer(0) ) ;
Integer x = ( Integer ) myIntList.iterator().next() ;
```

De ce sa nu folosim pur si simplu un tip general de date (e.g. Object in Java)?

```
List myIntList = new LinkedList () ;
myIntList.add( new Integer(0) ) ;
Integer x = ( Integer ) myIntList.iterator().next() ;
```

... pentru ca pot aparea erori si nici codul nu este foarte clar

```
myIntList.add( new Player () ) ; // compilatorul nu previne asta
```

Daca in schimb folosim generics, compilatorul face exact asta pentru noi:

```
List <Integer> myIntList = new LinkedList <Integer>() ;
myIntList.add( new Integer (0) ) ;
myIntList.add( new Player () ) ; // eroare la compilare
Integer x = myIntList.iterator().next () ; // nu mai e nevoie de
```

- La compilare se produce un singur cod in care tipul formal este substituit cu tipul general (Object in acest caz)
- Compilatorul verifica tipul obiectelor folosite in clasele / metodele generice
- Nu mai sunt necesare casturi

(Type Erasure)

Daca in schimb folosim generics, compilatorul face exact asta pentru noi:

```
List <Integer> myIntList = new LinkedList <Integer>() ;
myIntList.add( new Integer (0) ) ;
myIntList.add( new Player () ) ; // eroare la compilare
Integer x = myIntList.iterator().next () ; // nu mai e nevoie de
```

- La compilare se produce un singur cod in care tipul formal este substituit cu tipul general (Object in acest caz)
- Compilatorul verifica tipul obiectelor folosite in clasele / metodele generice
- Nu mai sunt necesare casturi

(Type Erasure)

Type Erasure: Orice program cu generics poate fi convertit intr-un program echivalent fara tipurile parametrizate si cu casturi in locurile potrivite

alexandru.olteanu@upb.ro

Type Erasure: Orice program cu generics poate fi convertit intr-un program echivalent fara tipurile parametrizate si cu casturi in locurile potrivite

Type Erasure: Practic run-time nu va stii tipul formal, dar compilatorul are grija sa impuna restrictiile date de template

Genericitatea este un mecanism prin care tipurile folosite in definirea claselor / interfetelor si metodelor sa fie parametrizate.

Acest mecanism se intalneste, in principiu, in mai multe limbaje OOP:

- generics in Java, C# si Objective-C
- templates in C++

alexandru.olteanu@upb.ro

CLASE GENERICE

alexandru.olteanu@upb.ro

Folosirea tipului formal in definirea si instantierea claselor

```
public class Erasure <T> {
    private T obj;
    Erasure (T o) { obj = o; }
    T getObj () { return obj ; }
}
...
public static void main ( String [] args ) {
    Erasure <Integer> test = new Erasure <Integer>(10) ;
    System.out.println ( test.getObj () ) ;
}
```

```
public interface Map <K,V >{
  static interface Map.Entry <K,V >;     // tipuri multiple
  Set < Map.Entry <K,V> > entrySet () ; // tipuri imbricate
  ...
}
```

Folosirea tipului formal in definirea si instantierea claselor

```
public class Erasure <T> {
    private T obj;
    Erasure (T o) { obj = o; }
    T getObj () { return obj ; }
}
...
public static void main ( String [] args ) {
    Erasure <Integer> test = new Erasure <Integer>(10) ;
    System.out.println ( test.getObj () ) ;
}
```

Incercati sa decompilati o clasa cu javap (folosind -c). Observati Type Erasure.

Lista restrictii

Cannot Create Instances of Type Parameters

```
public class Erasure <T> {
  private T obj;
  Erasure () { obj = new T() ; } // does not compile
}
```

De ce?

Lista restrictii

Cannot Create Instances of Type Parameters

```
public class Erasure <T> {
  private T obj;
  Erasure () { obj = new T() ; } // does not compile
}
```

De ce?

... pentru ca nu ar avea sens sa instantiem Object, vrem doar ca referintele sa fie Object

Lista restrictii

Cannot Create Arrays of Parameterized Types

```
public class GenericsErasure <T> {
  private T[] objs ;
  Erasure () { objs = new T [10]; } // does not compile
}
```

De ce?

Lista restrictii

Cannot Create Arrays of Parameterized Types

```
public class GenericsErasure <T> {
  private T[] objs ;
  Erasure () { objs = new T [10]; } // does not compile
}
```

De ce?

... pentru ca ar insemna sa instantiem obiecte din tipul parametrizat

Este o lista de Strings o lista de Objects?

```
List <String > ls = new ArrayList <String >() ;
List <Object > lo = ls;
```

Este o lista de Strings o lista de Objects?

```
List <String > ls = new ArrayList <String >() ;
List <Object > lo = ls;
```

... nu, pentru ca daca compilatorul ar permite asta ar duce la erori... lo.add(new Object());

alexandru.olteanu@upb.ro

Si totusi ce pot face daca vreau sa definesc o metoda care sa ia ca parametru o lista de orice fel de elemente?

- nu pot folosi List (vezi slidul anterior)
- dar pot folosi wildcards (?)

```
void printList (List <?> l) {
  for ( Object e : l) { // e ok ?
    System .out. println (e) ;
  }
  l.add(new Object() ) ; // e ok ?
}
...
printList (ls);
```

Si totusi ce pot face daca vreau sa definesc o metoda care sa ia ca parametru o lista de orice fel de elemente?

- nu pot folosi List (vezi slidul anterior)
- dar pot folosi wildcards (?)

```
void printList (List <?> l) {
  for ( Object e : l) { // e ok ?
    System .out. println (e) ;
  }
  l.add(new Object() ) ; // e ok ?
}
...
printList (ls);
```

... e ok ca referinta sa fie Object (upcast), dar nu e ok ca tipul sa fie Object (pentru ca depinde de tipul parametrizat folosit la apel)

Tipurile formale si wildcards pot upper bounded (? extends T) si lower bounded (? super T):

```
void sayHello (List <? extends Person > l) {
    for ( Object e : l) {
        System .out. println (" Hello "+e. getName () ) ;
    }
}
```

```
public class Collections {
    static <T extends Comparable <? super T> >
    void sort (List <T> list ) {
        ... // o singura implementare
    }
}
```

Type Erasure cand folosim bounded wild cards:

```
public class Erasure <T extends Number > {
    private T obj;
    Erasure (T o) { obj = o; }
    T getobj () { return obj ; }
}
...
public static void main ( String [] args ) {
    Erasure < Integer > test = new Erasure < Integer >(10) ;
    System.out.println ( test.getObj () ) ;
}
```

Incercati sa decompilati o clasa cu javap

METODE GENERICE

alexandru.olteanu@upb.ro

Pentru metode, de ce sa nu supraincarcam metoda cu fiecare tip de date?

```
public class Collections {
  static boolean replaceAll (List < Integer > list , Integer old
    ... // o implementare
  }
  static boolean replaceAll (List <Player > list, Player oldVal
    ... // o alta implementare
  }
}
```

Pentru metode, de ce sa nu supraincarcam metoda cu fiecare tip de date?

```
public class Collections {
  static boolean replaceAll (List < Integer > list , Integer old
    ... // o implementare
  }
  static boolean replaceAll (List <Player > list, Player oldVal
    ... // o alta implementare
  }
}
```

... pentru ca am duplica cod (ceea ce e rau pentru ca... ?)

Pentru metode, de ce sa nu supraincarcam metoda cu fiecare tip de date?

```
public class Collections {
  static boolean replaceAll (List < Integer > list , Integer old
    ... // o implementare
  }
  static boolean replaceAll (List <Player > list, Player oldVal
    ... // o alta implementare
  }
}
```

... pentru ca am duplica cod (ceea ce e rau pentru ca... ?)

... pentru ca pot aparea buguri mai usor (cand avem de facut o modificare, trebuie sa o facem in mai multe locuri) si pentru ca este mai greu de testat (trebuie testate toate variantele in loc de una singura)

```
public class Collections {
  static boolean replaceAll (List < Integer > list , Integer old
    ... // o implementare
  }
  static boolean replaceAll (List <Player > list, Player oldVal
    ... // o alta implementare
  }
}
```

Daca in schimb folosim generics:

```
public class Collections {
    static <T>
    boolean replaceAll (List <T> list , T oldVal , T newVal ) {
        ... // o singura implementare
    }
}
```

De ce sa nu folosim pur si simplu Object?

```
// Metoda corecta
static <T> void arrayToCollection (T[] a, Collection <T> c) {
    for (T o : a) {
        c.add(o) ;
    }
}
// Metoda incorecta : de ce?
static void arrayToCollection ( Object [] a, Collection <? > c)
    for ( Object o : a) {
        c.add(o) ;
    }
}
```

La folosire, compilatorul deduce automat tipul formal:

```
static <T> void arrayToCollection (T[] a, Collection <T> c) {
    for (T o : a) {
        c.add(o) ;
    }
}
String [] sa = new String [100];
Collection <String > cs = new ArrayList <String >() ;

// T inferred to be String
arrayToCollection (sa , cs) ;
```

Putem sa avem metoda generica, chiar daca clasa nu este generica. Daca si clasa este generica, a nu se confunda tipul formal al metodei generice cu tipul formal al clasei

```
public interface Collection <E> extends Iterable <E> {
    boolean add(E e) ;
    boolean addAll ( Collection <? extends E> c) ;
    <T> T[] toArray (T[] a) ; // programmer controls the type
}
List < Integer > myList = new ArrayList < Integer >() ;
Integer [] myArray = myList.toArray (new Integer [ myList.size()
Number [] myArray = myList.toArray (new Number [ myList.size() ]
Object [] myArray = myList.toArray (new Object [ myList.size() ]
```

Putem sa avem metoda generica, chiar daca clasa nu este generica. Daca si clasa este generica, a nu se confunda tipul formal al metodei generice cu tipul formal al clasei

```
public interface Collection <E> extends Iterable <E> {
    boolean add(E e) ;
    boolean addAll ( Collection <? extends E> c) ;
    <T> T[] toArray (T[] a) ; // programmer controls the type
}
List < Integer > myList = new ArrayList < Integer >() ;
Integer [] myArray = myList.toArray (new Integer [ myList.size()
Number [] myArray = myList.toArray (new Number [ myList.size() ]
Object [] myArray = myList.toArray (new Object [ myList.size() ]
```

De ce nu? `<E> E[] toArray(E[] a)`

Putem sa avem metoda generica, chiar daca clasa nu este generica. Daca si clasa este generica, a nu se confunda tipul formal al metodei generice cu tipul formal al clasei

```
public interface Collection <E> extends Iterable <E> {
    boolean add(E e) ;
    boolean addAll ( Collection <? extends E> c) ;
    <T> T[] toArray (T[] a) ; // programmer controls the type
}
List < Integer > myList = new ArrayList < Integer >() ;
Integer [] myArray = myList.toArray (new Integer [ myList.size()
Number [] myArray = myList.toArray (new Number [ myList.size() ]
Object [] myArray = myList.toArray (new Object [ myList.size() ]
```

De ce nu? `<E> E[] toArray(E[] a)`

...pentru ca vreau sa pot scoate Number[] din List

Putem sa avem metoda generica, chiar daca clasa nu este generica. Daca si clasa este generica, a nu se confunda tipul formal al metodei generice cu tipul formal al clasei

```
public interface Collection <E> extends Iterable <E> {
    boolean add(E e) ;
    boolean addAll ( Collection <? extends E> c) ;
    <T> T[] toArray (T[] a) ; // programmer controls the type
}
List < Integer > myList = new ArrayList < Integer >() ;
Integer [] myArray = myList.toArray (new Integer [ myList.size()
Number [] myArray = myList.toArray (new Number [ myList.size() ]
Object [] myArray = myList.toArray (new Object [ myList.size() ]
```

De ce nu? `<T super E> T[] toArray(T[] a)`

Putem sa avem metoda generica, chiar daca clasa nu este generica. Daca si clasa este generica, a nu se confunda tipul formal al metodei generice cu tipul formal al clasei

```
public interface Collection <E> extends Iterable <E> {
    boolean add(E e) ;
    boolean addAll ( Collection <? extends E> c) ;
    <T> T[] toArray (T[] a) ; // programmer controls the type
}
List < Integer > myList = new ArrayList < Integer >() ;
Integer [] myArray = myList.toArray (new Integer [ myList.size()
Number [] myArray = myList.toArray (new Number [ myList.size() ]
Object [] myArray = myList.toArray (new Object [ myList.size() ]
```

De ce nu? `<T super E> T[] toArray(T[] a)`

...pentru ca as putea incerca sa scot un Integer[] din List<Number>

FOOD FOR THOUGHT

alexandru.olteanu@upb.ro

Why isn't `Collection.remove(Object o)` generic? Josh Bloch and Bill Pugh (some of the guys who worked on generification of the Collections Framework) refer to this issue in:

Java Puzzlers IV: The Phantom Reference Menace, Attack of the Clone, and Revenge of The Shift.

Cand o clasa extinde o clasa generica sau implementeaza o interfata generica si se ajunge la situatia de a avea o metoda 'suprascrisa' cu tip de rezultat diferit, compilatorul produce o metoda sintetica (care nu apare in cod si nu poate apelata explicit) numita bridge method.

Java Bridge Methods explained

Parametric polymorphism (...), allows a single piece of code to be typed "generically", using variables in place of actual types, and then instantiated with particular types as needed. Parametric denitions are uniform: all of their instances behave the same. (...) *B. Pierce, "Types and Programming Languages", MIT Press*

Why is C++ said not to support parametric polymorphism?